She explained that extending the `Window` interface illustrates interface declaration merging, a core TypeScript feature for augmenting existing types. She also noted that the `new` keyword before `YT.Player` indicates that `Player` is a constructor function. This matches the YouTube API requirement to instantiate the player with `new`, similar to how classes are instantiated in TypeScript.

She highlighted that the `onReady` property is a function type with a specific signature, and its event parameter is an object type containing a nested function. Jane marked `onYouTubeIframeAPIReady` as optional (`?:`) because the `useEffect` cleanup function deletes `window.onYouTubeIframeAPIReady`. This informs TypeScript that the property may not exist at runtime, preventing type errors after the component unmounts.

When asked about alternative solutions, Jane proposed leveraging the `@types/youtube` package, which provides pre-defined TypeScript types for the YouTube API. This eliminates the need for manual declaration merging and simplifies the code while ensuring full type safety. The package can be installed with the command: `npm install --save-dev @types/youtube`. The simplified code, shown listing 3.22, is located at `use-cases/YouTubeApiExample.tsx`. This example can be executed by running: `VITE_NAME=YouTubeApiExample npm run dev`.

Listing 3.22 Solution using the @types/youtube package

```
declare global {
  interface Window {
    onYouTubeIframeAPIReady?: () => void;
  }
}
const YouTubeApiExample: React.FC = () => {
  const playerRef = useRef<HTMLDivElement>(null);
  useEffect(() => {
    ...
    window.onYouTubeIframeAPIReady = () => {
      ...
      if (playerRef.current) {
        ...
          playerVars: {
            autoplay: YT.AutoPlay.AutoPlay ?? 1,
            mute: YT.Mute.Muted ?? 1,
```